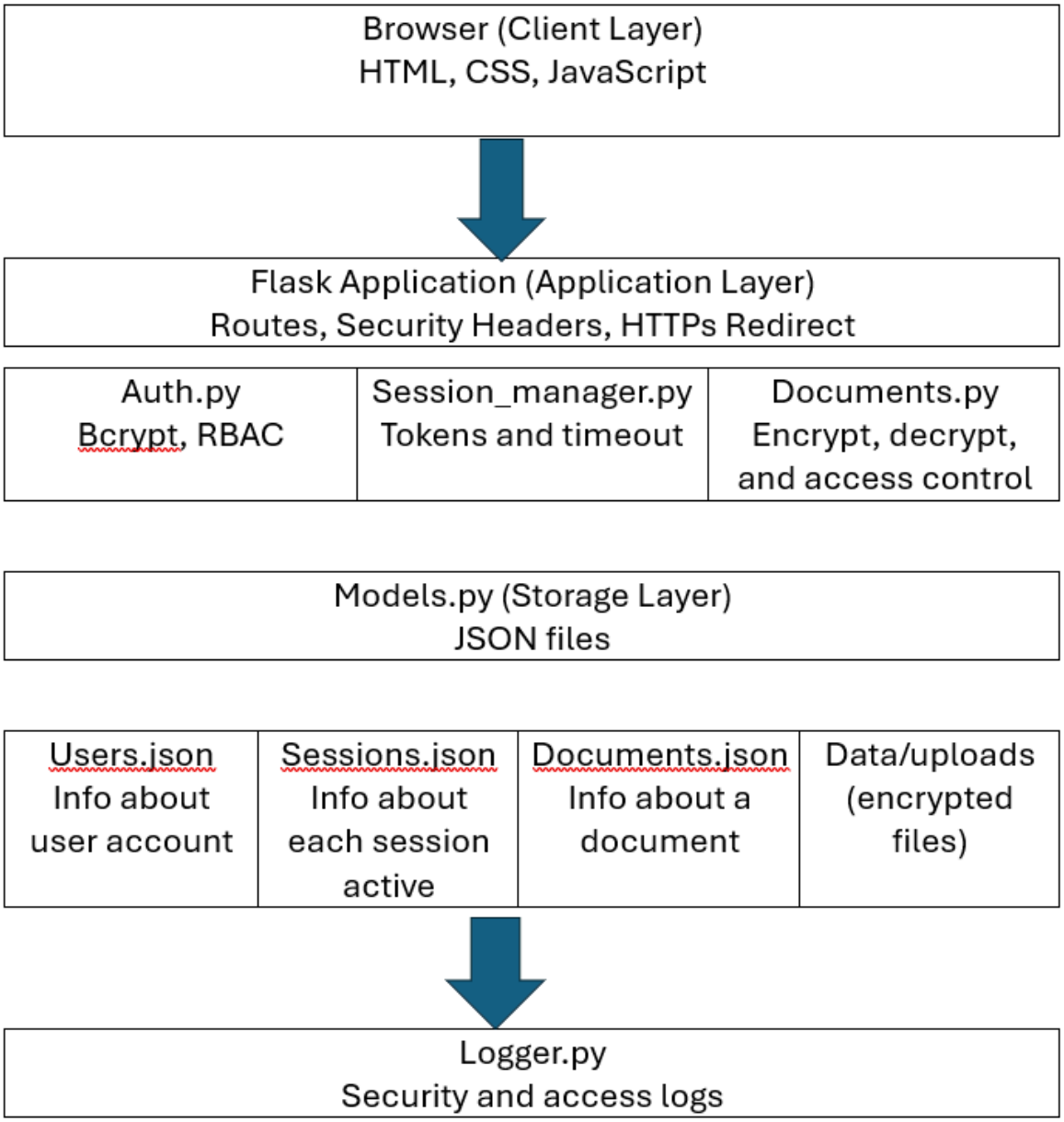
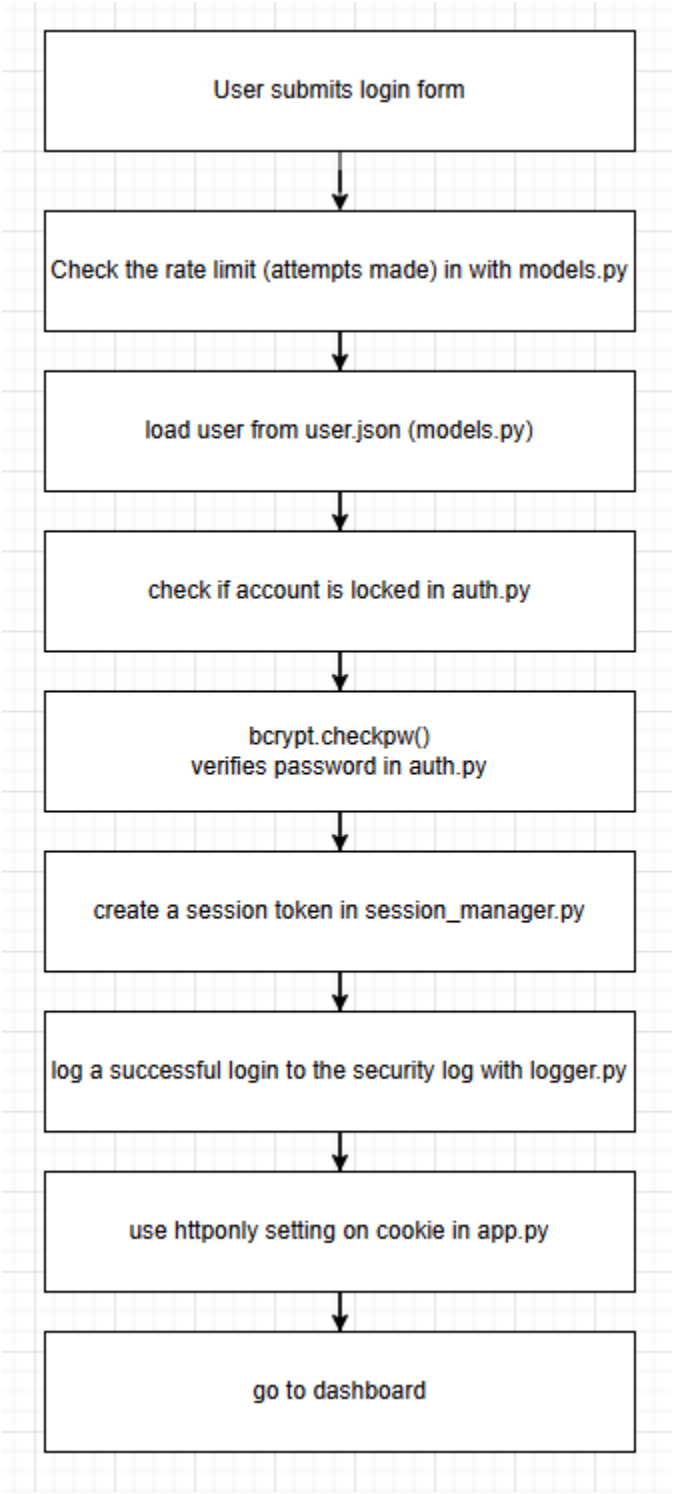
CS419 Project Security Design Document
Group Members: Matthew Ko and Michael Zdunek

A) Architecture Overview

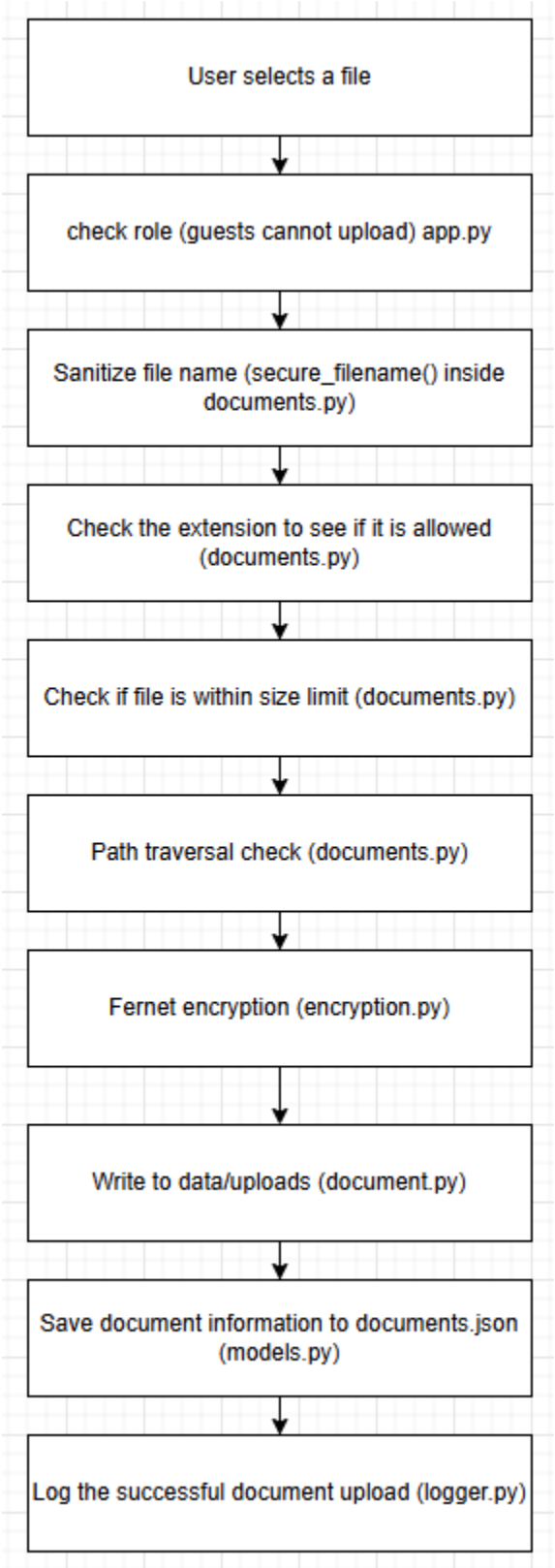
- System Architecture
    - The website has three layers which include the client, application, and storage.
    - Client layer includes the part where the user is interacting with the website through a browser. This layer uses HTML, CSS, and JavaScript. There are no logical operations run on this layer, instead it is only responsible for rendering the html templates and submitting information the client enters to the application.
    - The application layer uses Python and Flask along with other packages required to run the app. Along with the actual app.py, there are many other python files such as auth.py, session_manager.py, documents.py, encryption.py, logger.py, models.py, and lastly config.py.
        - auth.py handles the authentication of users and also the access control.
        - session_manager.py manages the session tokens.
        - documents.py handles any file and document operations.
        - encryption.py provides all of the encryption services.
        - logger.py handles logging of the events that happen such as access, successful, and failed events.
        - models.py provides data access to files
        - config.py contains the settings for the app
    - The storage layer includes json files to save information about sessions, documents, and users. There are also files for security logs and access logs.


Browser (Client Layer)
HTML, CSS, JavaScript
Flask Application (Application Layer)
Routes, Security Headers, HTTPs Redirect
Auth.py
Bcrypt, RBAC
Session_manager.py
Tokens and timeout
Documents.py
Encrypt, decrypt, and access control
Models.py (Storage Layer)
JSON files
Users.json
Info about user account
Sessions.json
Info about each session active
Documents.json
Info about a document
Data/uploads (encrypted files)
Logger.py
Security and access logs

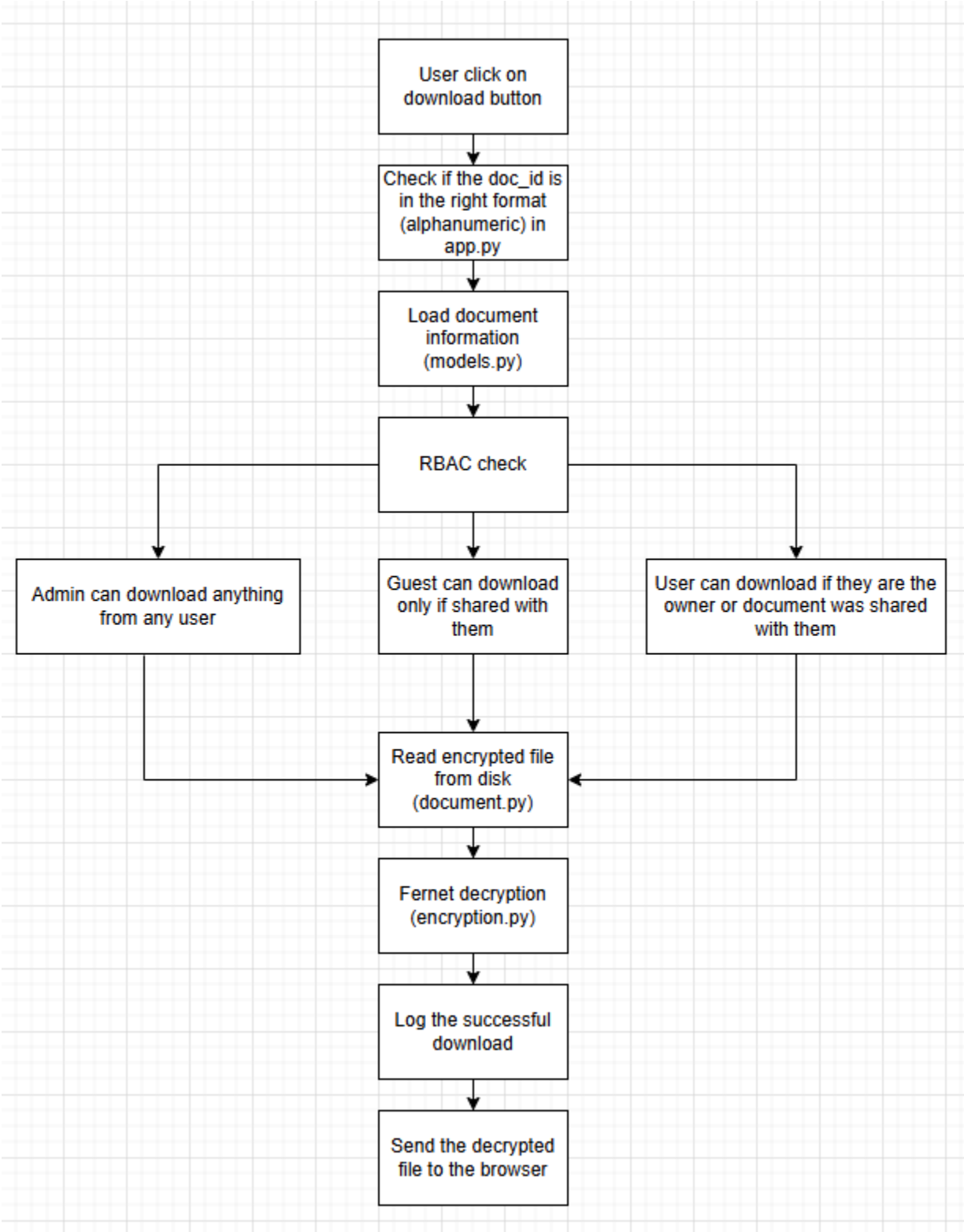
Authentication Flow:



Document Upload Flow:



Document Download Flow



Component Descriptions:

Component	**File**	**Responsibility**
Flask Application	app.py	Routes to different pages, security headers, HTTPS redirect, and logging access
Authentication	auth.py	Registration, login, bcrypt hashing, lock out, rate limiting, RBAC stuff
Session Management	session_manager.py	Handles secure tokens, validating them, timing out sessions, and destroying sessions
Document handling	documents.py	Encrypted upload, download, share, delete functions with correct access control
Encryption	encryption.py	Uses Fernet to encrypt, decrypt, and generate key
Data access	models.py	JSON access read and writes. Also keeps track of rate limit (attempts)
Security logging	logger.py	Methods to log security and access events and write them to security.log and access.log
Configuration	config.py	Settings for the application that you can change such as timeouts, limits, and file paths.

Technology Stack Justification

- Python/Flask was chosen because it was the recommended option for beginners in the project description. Allowed us to make a website that could interact with to upload, download, share, and delete files.

- Bcrypt was chosen to hash passwords because it is intentionally slow to make brute force attack even harder because it takes so much longer.
- Fernet was chosen because it implements symmetric encryption and provides authentication and confidentiality. If data is tampered with, it will detect it.
- File-based storage using JSON was chosen because of project directions and allowed us to easily save and read information from a file.
- secrets import was chosen because it is unpredictable and cryptographically secure compared to the “random” import

B) Threat Model

- **Asset identification**
    - User passwords
    - Encryption key
    - Uploaded documents
    - Session tokens
    - Usernames and emails
    - Filenames, who is sharing with who, owner
    - Security logs
    - Flask secret key
- **Thread enumeration**
    - Attacker obtains a user’s account information and shares files
    - Attacker obtains an admin account and promotes users from to higher privileges such as guest to user or user to admin
    - Attacker gains access to users.json and directly changes account roles
    - Attacker gains access to secret.key and decrypts documents
    - Attacker brute forces weak/common passwords
    - Attacker locks out a legitimate account by intentionally spamming passwords
    - Attacker uses known username and password from other leaked sites (credential stuffing)
    - Attacker gains access to access and security logs to tamper and hide their attack attempts
    - Attacker uses path traversal on file upload
- **Vulnerability assessment**
    - Data such as usernames, emails, sessions, and document info are stored in plaintext
    - Fernet key is also in the data directory, so if they gain access to that directory, they have the key as well.
    - Rate limiting is in memory instead of records in a file, so an attacker could wait for an application restart, or even cause one themselves if possible

- There is no password recovery function when logging in, only change password after the user has logged in. Because of this, if the user forgets their password, they are permanently locked out

- **<u>Attack Scenarios</u>**
  - Brute force login
    - Attacker repeatedly attempts to log in with a known username
    - Application blocks account after 5 failed attempts and blocks ip after 10 attempts with different usernames. Logs all attempts inside security log as well
  - Privilege Escalation
    - A guest or user tries to access the /admin page to see all documents (with full power) and promote other users to higher power (guest to user or user to admin).
    - require_role('admin') makes it so users that are not admin role returns a 403 error
  - Malicious file type upload
    - Attacker attempts to upload files with extensions that are not allowed
    - _safe_ext() checks the allowed file types then rejects it if it is not allowed
  - Path traversal
    - Attacker tries to move through directories with a filename such as ../../users.json
    - secure_filename() from the werkzeug import strips the input of dangerous characters and abspath checks and compares the path that it would go to and the intended path that it was supposed to go to. Rejects if it does not match with the base absolute path
  - Session token theft
    - Attackers want to access a users account without their username and password. The attacker tries XSS attack or network sniffing
    - XSS prevented from httponly=True inside app.py. JavaScript is not allowed
    - Packet sniffing is prevented by "secure=" inside app.py because it ensures it is transmitted over https so everything is encrypted.

- **Risk Prioritization**
    - Critical
        - Encryption key leaks: if someone got the key then they would have access to all documents
    - High
        - Brute force attacks
        - Session token theft
    - Medium
        - In memory rate limiting (resetting application resets the lockout records)
    - Low
        - XSS from file names: sanitize and Jinja make this unlikely
        - Path Traversal: secure_filename() and abspath check make this unlikely

C) Security Controls

A) User Authentication

- Control Description: User needs to register with a valid username, email, and strong enough password. Login requires the user to enter correct credentials and is protected from brute force attacks from lockout/rate limiting
- Implementation approach: Passwords are hashed using bcrypt with a cost factor of 12. The register_user() function in auth.py makes sure the inputs are valid with regular expressions before hashing. The login_user() function uses bcrypt.checkpw() to verify password and the plaintext password is never stored anywhere. The application also supports locking users out after several failed attempts.
- Testing methodology
    - Attempted to login with wrong password 5 times to ensure account locks
    - Ensured login attempts on locked account still fail with correct password
    - Verified lockout expires after 15 minutes
    - Ensured rate limiting stops attempts from same IP after 10 failed attempts
    - Verified passwords are hashed and stored in users.json
    - Rejects registration attempts with weak passwords
- Known Limitations
    - Rate limiting record is in memory and not in json files so it resets on application restart
    - Rate limiting can be avoided if attacker has multiple IP addresses
    - Account lockout (after 5 failed attempts) can lock out legitimate users if attacker purposely fails login attempts
- Mitigation strategies
    - Notify a user with an email if their account gets locked
    - Implement a CAPTCHA after several failed login attempts

B) <u>Access Control</u>

- Control Description: RBAC (Role based access control) is implemented with three roles which are admin, user, and guest. Each has different levels of permissions.
- Implementation approach: The require_auth() decorator function in auth.py checks that a valid user exists inside g.user before allowing access to routes that are protected. If there is not a valid user, it just redirects to the login page (/login). The require_role() decorator checks that the user's role matches the role that is required for the function, and returns a 403 error if they are not the correct role. Guests can only download files that were shared to them and nothing else. Users can upload, download, share, and delete their own files (also download files shared to them). Admin can upload, download, share, and delete ALL files, even if not shared to them. They can also promote and demote other accounts through the admin dashboard.
- Testing methodology
    - Attempted access to /admin as user and guest and verified it returns 403 error
    - Verified admin can perform all actions on any file regardless of ownership.
- Known Limitations
    - Viewer and editor role when sharing files don't actually do anything
    - Role info is stored in plaintext in users.json, so if attacker gains access to the json file, they can easily change the role
- Mitigation strategies
    - Add functionality to viewer and editor roles
    - Add integrity check to roles such as HMAC (hash based authentication code)

C) <u>Input Validation & Injection Prevention</u>

- Control Description: All user input is sanitized and checked before being processed and stored. Some functionality to filter out bad input include regular expressions and whitelisting (file extensions).
- Implementation approach: The sanitize() function in auth.py uses html.escape() on user input such as usernames and emails which gets rid of dangerous characters. Regular expressions validate the format of the usernames and emails as well. File uploads are checked by using the _safe_ext() and Config.ALLOWED_EXTENSIONS to make sure the file type is allowed. Path traversal is stopped by using secure_filename() from werkzeug and making sure the final absolute path starts with the intended base.
- Testing methodology
    - Attempting registration with <script>alert(1)</script> fails
    - Uploading file with .exe file fails and states "File type not allowed. Allowed: docx, jpeg, jpg, pdf, png, txt"
    - Attempted path traversal via filename with curl command: curl.exe -k -b "session_token=COOKIE" -F "file=@testfile.txt;filename=../../users.json"

https://127.0.0.1:5000/documents/upload. (Replace COOKIE with session cookie when logged in)
    - Attempted to trick app with https://localhost:5000/documents/../users.json/download url
- Known Limitations
    - File extension check only the extension but not the file contents
- Mitigation strategies
    - Possibly add antivirus scanning for uploaded files

D) <u>Encryption</u>

- Control Description: All documents are encrypted before being saved by using the Fernet symmetric encryption. Without the encryption key, attackers will not be able to read contents.
- Implementation approach: The EncryptionService class inside encryption.py generates a Fernet key on the first time it runs and saves it to data/secret.key. After this, every time it starts up it loads this key from this file. Every file that is uploaded is encrypted using the encryption.encrypt() method before being written to disk and decrypted using encryption.decrypt() when someone downloads a file. The files are stored with the .enc extension and use a UUID as a name. The file name is saved separately in documents.json. Transport encryption is handled using a self signed certificate with an ssl context, enforcing HTTPS in the enforce_https() function in app.py
- Testing methodology
    - Uploaded a text file that I know the contents of and checked the file inside the uploads folder and verified that the file has random contents
    - Checked that the original filename is not used to save as a file, instead has the UUID as a name and the original filename is in documents.json
    - Downloaded the same file through the app and verified that the content was the same that I uploaded
- Known Limitations
    - The Fernet key is stored in the same filesystem as the encrypted files, so if an attacker gets access to this location, they get access to the key and documents
    - The uploaded documents are encrypted but the other jsons such as users.json, sessions.json, and documents.json are stored as plaintext
- Mitigation strategies
    - Store the encryption key in a different location
    - Encrypt users.json, sessions.json, and documents.json

E) <u>Session Management</u>

- Control Description: User sessions are made when they successfully login, and sessions are destroyed when logging out or changing their password. Sessions expire after 30 minutes of inactivity. Each session also holds the user's IP address and agent for auditing and logging.
- Implementation approach: The SessionManager class in session_manager.py generates session tokens with secrets.token_urlsafe(32) which creates a

cryptographically secure random string. When logging in, app.py sets the token as a cookie with HttpOnly = True to prevent JavaScript access and SameSite = Strict for CSRF protection. Sessions are stored in sessions.json, and the username, creation time, last activity time, IP address, and user agent are held here. Every time session_manager.validate() is called (which is called in load_session_user(), then load_user uses load_session_user()), it checks if the session exists or if more than 30 minutes have passed since the last activity. If more than 30 minutes have passed, then the session is destroyed and the user is redirected to log in once again. When a user logs out, it destroys the session and removes it from sessions.json. If the user changes their password, it destroys ALL active sessions for the user and sends them back to the login page.

- Testing methodology
    - Logged in and verified a session token was set in the browser
    - Logged out and verified the session was removed from sessions.json
    - Tested the 30 minutes of inactivity and verified the session expires and sends me back to the login page
    - Changed password and verified all sessions are removed
- Known Limitations
    - Sessions are stored in plaintext, so attackers may gain access and be able to see the session cookie/token
- Mitigation strategies
    - Encrypt sessions.json

F) <u>Security Headers</u>

- Control Description: HTTP security headers are added to every response to tell the browser how to handle content and protect against common attacks like clickjacking, cross site scripting, and MIME sniffing.
- Implementation approach: Following the provided code, all of the security headers are placed in app.py and runs after every request using the app.after_request decorator. The decorator includes these headers: Content-security-policy, X-Frame-Options: DENY, X-Content_Type-Options:nosniff, X-XSS-Protection:1;mode=block, Referrer-Police:strict-origin-when-cross-origin, Permissions-Policy, and Strict-Transport-Security
- Testing methodology
    - Opened browser devtools with F12, went to network tab, clicked a request, then checked the response header to verify all headers are present
- Known Limitations
    - Content Security policy only allows scripts from our own server, but unsafe-inline tells the browser to run code that is in HTML which exposes a weakness because XSS scripts can use inline attacks.
- Mitigation strategies
    - Move all inline JavaScript to a file
    - Remove unsafe-inline from CSP

G) <u>Logging & Monitoring</u>

- Control Description: All security and access events are logged into json files called security.log and access.log. This allows us to have an auditing that helps us detect attacks.
- Implementation approach: The logger.py has two functions called log_security (that writes into security.log) and log_access which writes into log_access. The security.log file keeps track of security events such as successful and failed logins. The access.log file keeps track of HTTP methods such as GET and POST. Information the log_security writes to a json file include time, event type, user ID, IP address, user agent, and severity level (Info, warning, error, critical). The log_access information includes method, path, status code, user id, and IP address.
- Testing methodology
    - Logged in and checked a LOGIN_SUCCESS appeared in the security log
    - Attempted to log in with incorrect credentials and checked that LOGIN_FAILED appeared with a WARNING level.
    - Verified that locking an account with too many attempts shows ACCOUNT_LOCKED with ERROR level
    - Verified the log contains time, event, IP address, and user ID
- Known Limitations
    - All of the log files are in plaintext so if attacker had access to logs they could try to hide their tracks/attacks
    - There are no alerts for suspicious activity, so an attacker can keep attempting logins after every lockout until someone notices.
    - Log files will continue to grow and take up more and more space
- Mitigation strategies
    - Add an alert system for dangerous security events
    - Encrypt log files with encryption.py functionality

<u>D) Data Protection</u>

- Data Classification:
    - Highly Sensitive
        - User passwords
        - Fernet encryption key (data/secret.key)
        - Session tokens (sessions.json)
        - Flask secret key (Config.SECRET_KEY)
        - SSL private key (key.pem)
    - Sensitive
        - Uploaded Documents (data/uploads)
        - Usernames and email addresses (users.json)
        - Sharing relationships and document data (documents.json)
    - Internal
        - Security logs (logs/security.log)
        - Access logs (logs/access.log)

        - Filename and document ownership
- Encryption Methods
    - Data at Rest: Fernet Symmetric Encryption
        - Uploaded Documents are encrypted using the Fernet encryption from the cryptography import before being written to a file. Fernet uses AES-128 in CBC mode and HMAC SHA256 for authentication. The encrypt() and decrypt() methods in encryption.py handle this, and the files are stored with an .enc extension with UUID as its filename
    - Password Hashing: bcrypt
        - User passwords are always encrypted and hashed before being stored and never stored in plaintext. When a user first registers, auth.py generates a salt using bcrypt.gensalt(rounds = 12), then hashes the password with bcrypt.hashpw() with the salt. The cost of 12 means bcrypt performs 2^12 = 4096 iterations, making brute force attacks slow and difficult. The hash is then stored in users.json and then verified and checked when a user logs in with bcrypt.checkpw()
    - Data in Transit: TLS/HTTPS
        - All communication from browser to server and server to browser are encrypted using TLS and a self signed SSL certificate. Flask loads cert.pem and key.pem when the application first runs (if they exist). When not in the development environment, app.py redirects HTTP to HTTPS automatically.
    - Session Tokens:
        - Session tokens are generated with the secrets.token_urlsafe(32) and transmitted over only HTTPS and stored in cookies with HttpOnly = True and SameSite = Strict
- Key Management
    - Fernet Encryption Key
        - The Fernet Encryption key is generated the first time EncryptionService.__init__() runs inside encryption.py with Fernet.generate_key(). It is written into data/secret.key and loaded from there on every other start up afterward.
    - Flask Secret Key
        - The Flask secret key is generated when the application starts running with secrets.token_hex(32) inside config.py.
    - SSL Private Key
        - The SSL private key (key.pem) is generated when running the openssl command (same one that generates the self signed certificate) and stored inside the root directory.
- Secure Deletion Procedures
    - Deleting documents:
        - When a user or admin deletes a document, the delete_document() function inside documents.py removes every version of the encrypted file from storage using os.remove(). Then it removes the document entry from

the documents.json file using docs.pop(doc_id, None). If a file version is already missing from the disk, the FileNotFoundError is caught and skipped inside an except statement.

- Replacing a document when uploading
    - If a user uploads a file with the same name as a file that they already own, it replaces and updates the file contents. It deletes the old .enc file before saving the new file.
- Session Deletion
    - When a user logs out, session_manager.destroy() removes the session from the sessions.json file and the cookie is also deleted from the browser from response.delete_cookie('session_token'). When a user changes their password, the destroy_all_for_users() method removes ALL active session for that user from the sessions.json.